

practical-5

AIM: Implementation of a knapsack problem using dynamic programming.

Knapsack:

ALGORITHM:

Input: Number of item n , weights $w[i]$, Values $v[i]$, Capacity w
output: Maximum value and Selected items

Step1: start
Step2: Read $n, w, w[i]$ and $v[i]$
Step3: initialize $dp[i][0] = 0$ and $dp[0][j] = 0$.
Step4: For $i = 1$ to n do.
Step5: For $j = 1$ to w , do
Step6: if $w[i] \leq j$, then do.
$$dp[i][j] = \max(v[i] + dp[i-1][j - w[i]], dp[i-1][j])$$

Step7: Otherwise do, $dp[i][j] = dp[i-1][j]$
Step8: End the inner loop
Step9: End the outer loop
Step10: Set $max = dp[n][w]$
Step11: Set $j = w$ and trace $i = n$ to 1
Step12: if $dp[i][j] \neq dp[i-1][j]$ then item i is taken and $j = j - w[i]$
Step13: otherwise, item i is not taken
Step14: Display maximum value & selected items.
Step15: Stop.

CODE:

```
#include<iostream> using
namespace std;

class Knapsack { private:
    int w[10];    // weights
    int v[10];    // values
    int n;        // number of items
    int W;        // capacity int dp[10][100];
    // dp table

public:

    // — Get Input — void getInput() {
        cout << "Enter number of items : "; cin >> n;
        cout << "Enter knapsack capacity : "; cin >> W;
        for(int i = 1; i <= n; i++) {
            cout << "Item " << i << " weight and value : "; cin >> w[i] >>
            v[i];
        }
    }

    // — Build dp table — void solve() {

        // base case — row 0 and col 0 for(int i = 0; i <= n;
        i++) dp[i][0] = 0;
        for(int j = 0; j <= W; j++) dp[0][j] = 0;

        // fill table
        for(int i = 1; i <= n; i++) { for(int j = 1; j <=
            W; j++) {
                if(w[i] <= j) {
                    // take or not take dp[i][j] = max(
                    v[i] + dp[i-1][j-w[i]], dp[i-1][j]
                    );
                } else {
                    // cannot take item i dp[i][j] =
                    dp[i-1][j];
                }
            }
        }
```



```
    }
  }
}

// — Print dp table — void printTable()
{
  cout << "\nDP Table:" << endl; cout << "
    ";
  for(int j = 0; j <= W; j++) cout << j << " ";
  cout << endl;

  for(int i = 0; i <= n; i++) { cout << "i=" << i
    << " "; for(int j = 0; j <= W; j++)
      cout << dp[i][j] << " "; cout << endl;
  }
}

// — Find selected items — void findItems()
{
  cout << "\nSelected Items:" << endl; int j = W;
  for(int i = n; i >= 1; i--) { if(dp[i][j] != dp[i-
    1][j]) {
    cout << "Item " << i
      << " (w=" << w[i]
        << ", v=" << v[i]
          << ") TAKEN " << endl;
    j = j - w[i];
  } else {
    cout << "Item " << i
      << " (w=" << w[i]
        << ", v=" << v[i]
          << ") NOT taken " << endl;
  }
}
}

// — Show result — void
showResult() {
  cout << "\nMaximum Value = "
    << dp[n][W] << endl;
```

```
}  
};  
  
// — MAIN — int main() {  
  
    Knapsack k; k.getInput();  
    k.solve();  
    k.printTable(); k.showResult();  
    k.findItems();  
  
    return 0;  
}
```

Output:

```
Enter number of items: 4  
Enter knapsack capacity: 10  
Item 1 weight and value: 6  
4  
Item 2 weight and value: 5  
9  
Item 3 weight and value: 10  
7  
Item 4 weight and value: 5  
7
```

DP Table:

	0	1	2	3	4	5	6	7	8	9	10
i=0	0	0	0	0	0	0	0	0	0	0	0
i=1	0	0	0	0	0	0	4	4	4	4	4
i=2	0	0	0	0	0	9	9	9	9	9	9
i=3	0	0	0	0	0	9	9	9	9	9	9
i=4	0	0	0	0	0	9	9	9	9	9	16

Maximum Value = 16

Selected Items:

```
Item 4 (w=5, v=7) TAKEN  
Item 3 (w=10, v=7) NOT TAKEN  
Item 2 (w=5, v=9) TAKEN  
Item 1 (w=6, v=4) NOT TAKEN
```

Time Complexity:

Time Complexity:

Average -

Best Case:

```

For i = 1 to n
  For j = 1 to n
    For k = 1 to w
      if (w[i] = j)
        dp[i][j] = max()
      else
        dp[i][j] = dp[i-1][j]
  
```

$T(n) = n \times w$
 $= nw$

Best Case

$$T(n) = O(n)$$

For i = 1 to n.

For j = 1 to w.

dp[i][j]

$$T(n) = n \times w$$

$$= nw$$

$$T(n) = O(nw) \text{ or } O(n)$$

Worst Case:

For i = 1 to n:

For j = 1 to w.

dp[i][j] = max()

$$T(n) = n \times w$$

$$= nw$$

$$T(n) = O(nw) \text{ or } O(n)$$

Conclusion: Thus, the 0/1 Knapsack problem was successfully implemented using Dynamic Programming. The algorithm finds the maximum possible value within the given capacity and also identifies the selected items. The dynamic programming approach avoids repeated calculations and efficiently solves the problem with a time complexity of $O(n \times W)$.